A Centralized AI Operations Platform

Practical Technical Guide & System Overview

AI Health Check

ARTI-409-A · AI Systems & Governance · Capstone

"One schema. One audit chain.
One human-in-the-loop contract."

Team: Jack · Sakir · Osele · Jeewanjot

April 2026

Table of Contents

Click any entry to jump to the section.

1. Introduction

2. Problem We Are Solving

3. Solution Overview (Big Picture)

4. System Overview (Simple Architecture)

5. Core Modules

6. End-to-End Workflow

7. How AI (LLM) is Used Safely

8. Data Flow (Simple Explanation)

9. Testing & Reliability

10. Governance & Risk Handling

11. Key Strengths of the System

12. Limitations (Honest Section)

13. Future Improvements

14. Quick Summary (Revision Section)

Bonus — How to Explain This System in 1–2 Minutes

1. Introduction

What this system is

The AI Health Check Platform is a single web application that gives a team one place to watch every AI service they run in production. Instead of piecing together five different tools to monitor quality, investigate problems, and prove compliance, the team uses one app that covers the full lifecycle.

Think of it as a control room for a fleet of AI services. Each service is a product that uses a language model under the hood. The platform keeps a live view of how those products are behaving, spots when something goes wrong, and guides the team through fixing it — all while keeping a tamper-evident record that an auditor can trust.

Who it is for

- Engineering teams running AI features in production and needing daily visibility.
- Technical leads responsible for AI quality and incident response.
- Compliance, governance, and audit teams who need evidence trails.
- Non-technical stakeholders who want to understand AI system health at a glance.

Why it is important

AI systems fail in ways that traditional monitoring does not catch. Servers can be 100% up while the model gives wrong answers. Users notice before anyone else. Investigations happen in chat threads and disappear. Audit evidence is scattered across screenshots and exports.

This platform closes all four of those gaps with one application, one database, and one approval contract that covers every AI action.

2. Problem We Are Solving

Why AI systems fail in real organizations

AI-powered products behave differently from normal software. The code may be healthy and the servers may be reachable, yet the actual answers the AI gives can silently decay. The team only finds out when a user complains or a report looks wrong.

The four chronic gaps

- **No shared runtime view —** teams track whether the server is up but cannot see whether the AI is behaving. Uptime is not behaviour.
- **Drift is caught by users, not tools —** answer quality slowly gets worse over weeks. Without a detector, the first signal is an angry customer.
- **Investigations live in chat —** when something breaks, the team diagnoses it in Slack, and the knowledge disappears. No tracked workflow, no permanent record.
- **Compliance evidence is stitched from many tools —** audit-time means exporting from five products and hoping nothing is missing. Silent gaps are the failure mode regulators care most about.

> **The short version**
> Traditional monitoring tells you the system is reachable. It does not tell you the AI is giving good answers, that humans reviewed every AI decision, or that the record is tamper-proof. That is the gap this platform closes.

3. Solution Overview (Big Picture)

What the platform does

The platform brings four concerns — registering services, monitoring them, investigating incidents, and proving compliance — into one application. Every action goes through the same approval logic, writes to the same audit chain, and is stored in the same database.

How it solves the problem

- **One registry —** every AI service has one record that owns its metadata, owner, environment, and sensitivity label.
- **Continuous evaluation —** the platform regularly tests every service with a small exam and scores the results. Quality is measured, not guessed.
- **Structured incidents —** when a problem appears, a tracked incident is opened, the AI drafts a summary, and a human reviewer approves it before it becomes official.
- **Tamper-evident log —** every action creates an audit row that is cryptographically linked to the one before it. Any alteration is detectable.

Key idea: the AI Operations Control Room

The platform is a single-pane control room. An operator glances at the Dashboard, sees traffic-light-style indicators across the entire AI fleet, and clicks into any red item to investigate. Every step they take — registering a service, running an evaluation, approving an AI-drafted summary, exporting a compliance report — happens in the same app and leaves a signed receipt.

What the live demo carries

The running demo database carries three registered AI services, five tracked incidents, seventeen evaluation runs, sixty-five audit entries, and two hundred and ninety-two scheduled ping logs. That is enough data for every chart to have shape and every workflow to have a story to tell.

> **One-line thesis**
> One schema, one audit chain, one human-in-the-loop contract — across monitoring, evaluation, triage, and governance.

4. System Overview (Simple Architecture)

The platform has four layers. Each layer does one job. A user request flows from the top down and a response flows back up.

The four layers

Frontend — the user interface

A modern web application built with React. It is what the team actually sees and clicks. Nine pages cover every module of the system: Dashboard, Services, Evaluations, Incidents, Governance, Settings, and so on. The interface is designed for a glance — numbers and charts update automatically without needing to refresh.

Backend — the logic and rules

A Python application built with FastAPI. Nine routers split the API along module boundaries (services, dashboard, evaluations, incidents, maintenance, audit, users, export, auth). The backend receives requests from the frontend, checks who is allowed to do what, runs the business logic, and writes to the database. All AI calls are routed through one central function, so every safety rule is enforced in one place.

Database — the storage

A SQLite database keeps every record across fourteen related tables: services, evaluations, incidents, maintenance plans, audit entries, users, and so on. Foreign keys are enforced at every relationship, and the audit log is append-only, protected by database triggers so rows cannot be altered or deleted.

LLM integration — the AI brain

The platform talks to Anthropic's Claude through one pipeline, but with a two-tier model economy. Claude Sonnet 4.6 plays the actor role — the service under test, incident summaries, dashboard insights, and compliance narratives. Claude Haiku 4.5 plays every judge role — factuality scoring, hallucination detection, and an independent prompt-injection classifier that backs up the regex tripwire. Using a smaller, differently trained model for the judges partially breaks the "model scoring itself" correlation and keeps the judge layer cheap. Every call passes through the same six safety stages before and after, and every call is fully logged with per-model cost accounting.

Background jobs — the heartbeat

An in-process scheduler runs two recurring jobs. A health check pings every registered service every five minutes and records latency. An evaluation run executes every active non-confidential service's test cases through Claude every sixty minutes and writes the resulting drift signals. Both jobs can be disabled with a single environment flag for stable demos.

How a request flows (simple example)

1. The user clicks a button in the browser.
2. The frontend sends a request to the backend over HTTPS.
3. The backend checks the user's role and, if needed, calls Claude through the safe pipeline.
4. The backend writes the result and an audit entry to the database.
5. The backend sends the result back to the frontend.
6. The frontend updates the screen.

> **One rule worth remembering**
> Every crossing between layers is audited. You cannot move data through this system without a record of who moved it and why.

5. Core Modules

The platform is built from four modules. Each one handles a specific part of the AI lifecycle. They share the same database and the same approval rules, so they work together naturally.

Those four modules surface in the user interface as nine pages — Dashboard, Services, Evaluations, Incidents, Maintenance (inside the incident detail view), Governance, Auth, Settings, and Data Policy. Every page follows the same conventions for polling cadence, timestamps, error handling, and audit logging.

Module 1 — AI Service Registry

What it does

The registry is the phone book of the platform. Every AI service is registered once with its metadata: name, owner, environment (production, staging, development), the model it uses, its endpoint URL, and most importantly its sensitivity label (public, internal, or confidential).

Key features

- Create, read, update, and delete AI service records.
- Connection testing — the Ping button verifies that the endpoint is reachable and responsive.
- SSRF protection on every URL — the system refuses to test services that point to internal or dangerous addresses.
- Sensitivity labels that every downstream module respects.
- Search, filter, and view toggles for working with many services at once.

Why it matters

Without a registry, each module would need its own list of services and its own rules. With one registry, every other module (monitoring, incidents, governance) simply references a service by ID. Sensitivity labels propagate automatically, so a confidential service is treated with extra care everywhere it appears.

Module 2 — Monitoring & Evaluation

What metrics are tracked

- Latency — how long each call to a service takes (typical, slow, and worst values).
- Quality — how accurate and helpful the responses are (scored from 0 to 100).
- Error rate — how often quality falls below the acceptable threshold.
- Active alerts — problems that currently need attention.

What evaluation means

Evaluation means running a small exam against an AI service. The exam is a question bank of test cases — each with a question and an expected answer. The service replies to each question, and a separate, smaller judge model (Claude Haiku 4.5) scores the reply for factuality and hallucination. Using a different model from the one under test partially breaks the "model scoring itself" correlation, and the platform records which judge model produced each run so auditors can trace any score back to the exact grader that generated it. The run produces one overall quality score.

What drift detection is (simple explanation)

Drift is when quality gets worse over time. The platform detects drift in two ways. First, it checks whether the latest score dropped below a threshold (a sudden collapse). Second, it looks at a running window of recent scores and asks whether they are trending downward (a slow decay). Either kind raises an alert and shows up on the Dashboard.

> **Why two drift checks**
>
> Quality can fail in two ways. It can crash suddenly (a prompt got corrupted) or it can slide slowly over weeks. A threshold catches the first. A trend catches the second. A real platform needs both.

Module 3 — Incident & Maintenance

What happens when something goes wrong

When a drift alert fires, the operator clicks Create incident from the notification. An incident form opens, already pre-filled with the service, the severity, the symptoms, and a suggested triage category. The user confirms the details and submits. The incident is now a tracked record in the platform.

Incident creation and investigation

- Every incident has a lifecycle: Open, Investigating, Resolved, Closed.
- A triage checklist helps the operator categorise the likely cause (data issue, prompt change, model update, infrastructure, safety policy).
- The incident detail page has three zones: symptoms and checklist on the left, an AI-drafted stakeholder summary on the right, and a list of maintenance plans below.

LLM assistance with human approval

The platform can generate a draft summary of the incident using Claude. The checked triage boxes are fed to the AI so the summary is grounded in the operator's hypotheses. The draft appears with an orange banner that reads "Draft needs human approval." The operator reads the draft, types a reviewer

note of at least 20 characters explaining what they verified, and clicks Approve. Only then does the draft become an official summary.

The 20-character requirement is enforced at three layers: the browser refuses to submit a short note, the request layer (Pydantic) rejects it again, and the request handler re-verifies before promoting the draft. Each layer alone could be bypassed by curl or a stale tab; together they cannot.

Maintenance planning

Once the cause is known, the team plans the fix. A maintenance plan records the risk level, the rollback strategy, the validation steps, and the scheduled date. A plan also needs explicit approval before it counts as agreed. The same approval pattern is used everywhere AI content or risky changes need to be signed off.

> **Human-in-the-loop contract**
> The AI drafts. The human approves with a note. Every time. The AI never publishes a summary, approves a plan, or closes an incident on its own.

Module 4 — Governance & Compliance

Role-based access

Three roles cover every user. Admins can do everything, including changing roles and overriding confidentiality gates. Maintainers can do most operational work but cannot change roles or send confidential data to the AI. Viewers have read-only access.

- Admin — full control and governance authority.
- Maintainer — can register services, run evaluations, manage incidents.
- Viewer — can see everything but cannot change anything.
- Every denied request is itself logged, so attempted misuse is on the record.

Audit logs

Every action anyone takes — every login, every service change, every incident approval, every failed attempt — writes a row to the audit log. The log is searchable and filterable, and it cannot be edited or deleted.

Data policy

Confidential services cannot have their data sent to the external AI without an explicit admin override, and that override is itself audited. The system has a safety scanner that blocks prompts containing

injection patterns, detected PII, or toxic content. Both the input and the output of every AI call are scanned.

Compliance report export

An admin can pick a date range and export the whole story as a PDF or JSON report. The export bundles audit entries, incidents with their approvals and reviewer notes, and maintenance plans with their scheduled dates. Large exports include a warnings section that flags if any part was truncated — no silent gaps.

The integrity check

A single button verifies the entire audit log. The system walks every row, recomputes its cryptographic fingerprint, and confirms that every row is linked to the one before it. If anyone tampered with a row, the chain breaks at the exact point of tampering. Green banner means the record is trustworthy.

6. End-to-End Workflow

Here is the full story of the platform from a healthy starting point to a closed-out incident with an auditor-ready report. Every step uses modules and patterns introduced earlier.

1. Add the AI service. An admin opens the Services page, fills out a short form, and clicks Save. The URL is checked for safety, the service is registered, and an audit entry is written.
2. Run monitoring automatically. Every few minutes the platform pings the service and records its latency and health. The Dashboard shows the live values.
3. Evaluate performance on a schedule. Every hour the platform runs its test exam against each non-confidential service, scores the answers using an AI judge, and averages the results into one quality score.
4. Detect a problem. The new score drops below the acceptable threshold. The platform marks the run as drift and records an alert row in the same transaction.
5. Alert appears. Within 30 seconds the notification bell lights up and the Dashboard shows the drift in red.
6. Create an incident from the alert. The operator clicks Create incident on the alert. The incident form opens with all the fields already filled from the alert details.
7. Investigate with the checklist. The operator ticks the triage categories they think are most likely (for example, data issue and prompt change) and clicks Generate draft. The AI reads the symptoms and the checklist and writes a draft stakeholder summary.
8. Human approves the draft. The operator reads the draft, types a reviewer note of at least 20 characters explaining what they verified, and clicks Approve. The draft becomes the official summary with the operator's name and note attached.
9. Plan the maintenance. The team adds a plan with the risk level, rollback, validation steps, and scheduled date. The plan is then formally approved.
10. Log every step. From the first click to the final approval, every action has been writing to the audit chain. Nothing is lost.

Then the auditor arrives

1. The auditor gives a date range.
2. The admin opens Governance, picks the range, and clicks Export as PDF.
3. The admin also clicks Verify integrity. A green banner confirms the record is intact.
4. The admin hands the PDF and a screenshot of the green banner to the auditor. That is the artifact.

Why this is powerful

The whole story — from the failure to the fix — is one continuous record in one app. There is no switching between tools, no copying screenshots, and no gaps for the auditor to worry about.

7. How AI (LLM) is Used Safely

Where AI is used

- Judge scoring — a cheaper, smaller model (Claude Haiku 4.5) grades the answers from a service under test.
- Hallucination detection — the same Haiku judge checks whether the service fabricated anything.
- Prompt-injection classifier — Haiku runs as a second layer over the regex tripwire on every input, catching paraphrased or novel injection attempts the regex misses.
- Incident summaries — Sonnet drafts a stakeholder-friendly write-up of the problem and root causes.
- Dashboard insights — Sonnet writes short, plain-language insights for trends the team should notice.
- Compliance reports — Sonnet drafts the narrative section of a compliance export.

The two-tier model economy

The platform uses two Anthropic models, each with a clearly defined role. Claude Sonnet 4.6 is the actor — it plays the service under test and writes every synthesis output. Claude Haiku 4.5 is the judge and the injection detector — it grades factuality and hallucination on every evaluation, and it provides the LLM layer of the input safety scan. Per-model pricing is recorded on every row so the cost ledger stays honest when a Haiku call and a Sonnet call sit side by side.

- **Why two tiers —** different size and training emphasis on the judge partially breaks the “model scoring itself” correlation, and Haiku is roughly a third the cost of Sonnet, so running a second layer on every input is affordable.
- **Why one vendor today —** one SDK, one authentication model, one rate-limit rhythm; swapping a second vendor in is a single configuration change because every AI call already flows through one function.
- **Honest caveat —** two tiers are not an ensemble. Both are still Anthropic models. A true cross-vendor judge (OpenAI or Gemini) is a documented next step on the roadmap.

What AI helps with

AI speeds up reading, summarising, and drafting. It scores an answer in seconds, writes a stakeholder update in under a minute, and explains a compliance posture in plain language. None of this replaces the team’s judgement — it just removes the blank page.

Why human approval is required

AI can hallucinate, misinterpret context, or generate plausible-sounding but wrong content. Every AI-generated artifact that goes into the record (incident summaries, compliance narratives, dashboard insights) must be reviewed by a named human. Approval requires a short note explaining what the reviewer verified. The draft is never promoted to official without that step.

The six-stage safety pipeline

Every AI call passes through six stages. Any stage can refuse the call before a single word reaches the model.

1. Sensitivity gate — confidential services require an admin override.
2. Input safety scan (two layers) — the prompt is checked first by a regex tripwire for known injection patterns, personal information, toxic content, and length, and then by a Haiku-based LLM classifier that catches paraphrased or novel injection attempts the regex cannot see. The two layers are merged conservatively: whichever layer returns the higher risk wins, so neither can silence the other. The classifier is fail-open — if it errors or times out, regex stays authoritative and the actor path is never blocked by an injection-checker outage.
3. Budget reserve — the call must fit inside the daily and per-minute budgets; concurrent calls cannot collectively overspend. The platform enforces sixty calls per minute globally and forty per user per minute — sized for the two-tier architecture's four Claude calls per user request (one actor, one injection classifier, two judges) — alongside five US dollars per day and twenty-five US dollars per month. Cost accounting is per-model, so a Haiku row at one dollar per million input tokens does not get priced at Sonnet's three dollars.
4. API call — the actual request to Claude (Sonnet for the actor, Haiku for judges and injection classifier), with automatic retry on transient failure.
5. Output safety scan — the response is checked for accidental PII before it leaves the backend.
6. Persist and audit — the call is saved with its cost, tokens, latency, the model that handled it, and any safety flags, and an audit entry is written.

> **The structural guarantee**
>
> There is exactly one place in the code that talks to the AI. Every feature that uses AI enters through that one function. Change a safety rule there, and every feature inherits it immediately. The only exception is the injection classifier itself — it enters through a sibling function that skips the input scan, because if it re-entered the scan it would loop forever trying to classify its own prompt. Every other feature routes through the full six-stage path.

8. Data Flow (Simple Explanation)

Data moves through the platform in a predictable path. Understanding that path helps explain where safety, audit, and approval checks sit.

Input → Processing → Storage → Display

1. Input — the user clicks a button in the web interface or fills out a form. The action is sent to the backend over HTTPS with an authentication token.
2. Processing — the backend validates the request, checks the user's role, applies safety rules, and (if needed) calls Claude through the safety pipeline.
3. Storage — the result is written to the database along with an audit entry. The audit entry is cryptographically linked to the previous one.
4. Display — the backend sends the result back to the frontend, which updates the screen. Dashboards update automatically on a refresh cycle.

Example: the full flow of one incident approval

- The reviewer clicks Approve in the browser.
- A modal asks for a note of at least 20 characters.
- The reviewer types the note and submits.
- The backend checks the note length again (because client-side rules can be bypassed).
- The backend flips the draft to an official summary and records the reviewer, the timestamp, and the note.
- An audit entry is written. Its cryptographic fingerprint links to the previous entry.
- The page refreshes and shows a green "Approved by" banner.

> **Where the trust lives**
> Every important check happens in the backend, not just the browser. Anyone with curl can bypass the UI — so the real rules live below the UI.

9. Testing & Reliability

Basic testing approach

The platform has a suite of automated tests that runs every time code is changed. Each test is a small, fast check — “does this feature still behave correctly?” If any test fails, the build fails and the change cannot ship.

- Approximately one hundred and fifty-eight tests across nineteen test files, covering authentication, services, evaluations, incidents, governance, drafts, per-model pricing, the two-layer safety scanner, the injection classifier, probe liveness, judge routing, and the AI pipeline end to end.
- Coverage is measured and must stay above a sixty-five percent floor that fails the build if it drops; the actual coverage today is around seventy-eight percent.
- The full suite runs in under a minute because tests use an in-memory database and a simulated AI client — no network calls, no cost.
- The codebase is around eight thousand lines of backend Python, six thousand lines of React, and three thousand lines of tests.

Tests that prove the tricky parts

- **Concurrency test —** twenty simulated callers hit the budget reservation at the same moment with the rate limit set to five; the platform must let exactly five through and reject the rest.
- **Integrity test —** the test tampers with the audit log, then checks that the verify function detects the break, and that foreign-key constraints and monotonic timestamps still hold.
- **Parser test —** the strict judge-score parser is fed malformed AI responses; only clean numbers are accepted, everything else is marked as a refusal and excluded from aggregates.
- **SSRF test —** ten URL families are fed through the safety validator (cloud metadata, RFC1918, IPv6 loopback, DNS-rebinding) to prove dangerous addresses are blocked and legitimate ones are allowed.
- **Auth test —** login success, login failure, and account lockout after repeated failures are all covered; every attempt is audited.
- **Compliance test suite —** twenty-eight tests covering hash-chain integrity, tamper detection, strict date parsing on exports, audit-log timezone correctness, and the rule that every export itself writes an audit entry.
- **Two-layer safety test —** six tests that prove the regex tripwire and the Haiku LLM classifier run independently, their risk scores merge via max-not-min, and a classifier outage leaves the regex layer authoritative (fail-open).

- **Injection-classifier test —** six tests for the Haiku-based prompt-injection detector: true positives on known injection phrasings, true negatives on benign prompts, and graceful fail-open when the classifier errors.
- **Per-model pricing test —** five tests that verify Sonnet rows are priced at three dollars per million input tokens and Haiku rows at one dollar, with an unknown-model fallback that warns once and falls back to Sonnet rates so cost is never silently under-counted.
- **Judge-routing test —** four tests that prove the judge calls are routed to the Haiku model, not the Sonnet actor, and that the EvalRun row records the exact judge model used.
- **Probe-liveness test —** five tests that prove scheduled health checks treat 4xx responses as reachable (many AI endpoints are POST-only and return 405 to anonymous GETs) and only count 5xx and network errors as failure.
- **Core-path test —** two tests that prove the public API call path always runs input safety, while the core path (used by the injection classifier itself) skips it — the recursion-prevention contract is enforced by tests, not trust.

Error handling

Every user-facing action handles failure. A service that does not respond records a failed connection and keeps the dashboard honest. An AI call that fails retries automatically up to twice before surfacing a clear error. Budget exhaustion or safety refusals return a clean message, not a cryptic stack trace.

Conventions every module follows

One of the quiet strengths of the platform is that every module behaves the same way. A reviewer who learns the conventions on one page can predict what will happen on every other page. The following rules are enforced across all nine pages.

- Timestamps are stored in UTC and rendered in the viewer's local time with a tooltip showing the original zone.
- List endpoints that return service-scoped data are filtered through one shared helper so the environment tabs (Dev, Staging, Production) work identically everywhere.
- Every page that shows live data polls at a right-sized cadence (typically thirty to sixty seconds) and displays a visible "updated N seconds ago" indicator with a one-shot pulse animation.
- Every parent-child relationship with a required foreign key cascades deletes so the database can never hold orphan rows.
- Every user-provided text field has a maximum length, enforced in the schema layer.
- Every mutation — create, update, delete, approve, export — writes an audit entry through the same helper.

- Every approval endpoint is idempotent: a second click returns a 409 Conflict so the original approver's attribution is never overwritten.
- Every form uses inline errors and toast notifications, not blocking alert dialogs. The browser's native window.prompt and window.confirm have been replaced with shared Modal-based dialogs everywhere the user types a reviewer note or confirms a destructive action.
- Every trend chip (Improving, Stable, Worsening) is colour-matched to the verdict semantics, so green always means good and red always means bad.
- Every loading, approving, or verifying state exposes an accessibility marker (aria-busy, aria-live) so screen readers follow along.
- Every KPI tile on the Dashboard exposes a structured tooltip (title, plain-English body, data-source / window line) so "Error Rate means quality drift, not HTTP failures" is visible on hover rather than buried in the docs.
- A persistent Notifications drawer in the top bar surfaces new drift alerts across sessions, and a DateTimeField component keeps scheduling inputs consistent between Incidents, Maintenance, and Evaluations.
- The audit log on the Governance page now renders a volume-stats strip (events, actors, last 24 h, top action), categorised action pills, and human-friendly target labels — the same raw rows, rendered for compliance reviewers instead of engineers.

Why reliability matters here

This is a governance tool. If the record cannot be trusted, the tool has no value. Every claim the platform makes — "AI calls are budget-limited," "the audit log is tamper-evident," "refusals are not counted as failures" — is backed by a specific test. That is what turns a demo into something you can defend in front of an auditor.

10. Governance & Risk Handling

Data privacy

Confidential services are isolated from the AI by default. Sending confidential data to an external model requires an admin to pass an explicit override, and the override itself is recorded. The safety scanner checks every prompt for emails, phone numbers, and SSNs, and scans every response for accidental PII leakage before it reaches the user.

AI risks addressed

- **Hallucination —** every AI-generated artifact is a draft until a human reviews it and adds a note. The draft is never the official version.
- **Drift —** threshold and trend detection catch both sudden collapse and slow decay of quality.
- **Prompt injection —** a two-layer defence at the input stage. Fifteen regex patterns catch known injection phrasings, and a Haiku-based LLM classifier provides an independent second opinion that catches paraphrased or novel attempts. The two layers merge conservatively (max risk wins), and the classifier fails open so an outage never unblocks an actual injection. The reviewer note contract Sakir demonstrated is the third backstop — anything that slips past both scanners still needs a human approver to sign off.
- **Data leakage —** output is scanned before it leaves the backend, so sensitive content is redacted or refused.
- **Abuse of privilege —** role-based access enforces who can do what. Every denied action is logged as an attempt.
- **Runaway spend —** daily and per-minute budgets are enforced atomically so even a surge of concurrent requests cannot exceed them. Rate limits are sixty calls per minute globally and forty per user per minute — sized for the two-tier architecture's four Claude calls per user evaluation request — with five US dollars per day and twenty-five per month as the dollar cap.

How the system reduces risk

Defence in depth. No single safeguard carries the whole weight. The approval contract is triple-gated — the browser checks the reviewer note length, the request boundary (Pydantic) checks it again, and the request handler re-verifies before promoting the draft. The audit log is append-only both by design and by database trigger. The AI pipeline runs six safety stages. If any one defence fails, the others catch the rest.

11. Key Strengths of the System

- **Unified platform —** monitoring, evaluation, incidents, and governance in one application with one database.
- **Governance-first design —** audit trails and human approvals are not bolted on; they are the skeleton of the app.
- **Human-in-the-loop contract —** the AI drafts, the human approves with a note, every time. Same contract for incidents, dashboard insights, and compliance reports.
- **Tamper-evident audit —** every action is cryptographically chained. A single button proves the chain is intact.
- **Single AI entry point —** every AI call, from every feature, goes through one function. Safety rules are enforced in one place.
- **Concurrency-safe budgets —** even twenty simultaneous requests cannot collectively exceed the daily cap. Proven by a test.
- **Auditor-ready exports —** pick a date range, download a PDF, hand it over. No stitching, no screenshots, no silent truncation.
- **Defence in depth —** every critical rule is enforced in at least two layers. No single point of trust.
- **Convention parity across every page —** a reviewer learns the timestamp, polling, audit, and cascade-delete conventions once and can predict the same behaviour on every one of the nine pages. No surprises.
- **Honest scope —** the platform does not pretend to be a multi-tenant enterprise product. It is a reference implementation that maps cleanly onto production technologies.

12. Limitations (Honest Section)

This is a capstone reference implementation. It is built to show that the governance model works end to end, not to be a ready-to-deploy enterprise product. The following simplifications are deliberate.

- **SQLite, not Postgres —** the database is a single file. A real deployment would use Postgres with full migrations. The schema is portable — the swap is a configuration change.
- **In-process scheduler —** background jobs run inside the same process. A real deployment would use a distributed worker queue such as Celery or RQ. The platform boundary is a drop-in replacement.
- **Single-vendor AI judge —** the judge is Claude Haiku 4.5 — a different size and training emphasis from the Claude Sonnet 4.6 actor, which partially breaks the “model scoring itself” correlation. But both are still Anthropic models. A true cross-vendor ensemble (an OpenAI or Gemini second judge) is a documented next step.
- **No multi-tenancy —** every user in the database is in one organisation. A multi-tenant system would add tenant scoping on every query.
- **No SSO or SAML —** authentication is username and password with lockout. A real deployment would integrate with a corporate identity provider.
- **Simulated data for the demo —** several services, evaluations, and drift scenarios are seeded so the demo is reliable. In a real system this data would grow organically over time.
- **AI costs are simulated locally —** token costs are computed from a pricing table. Real billing would reconcile against Anthropic’s own invoice.
- **No row-level access control —** role-based access decides what actions a user can take, not which services they can see. Per-team isolation is a documented next step.
- **Role-change UI uses a browser confirm dialog —** changing a user’s role still pops a native confirm() rather than the shared modal. The underlying action is fully audited and RBAC-gated; the cosmetic fix is on the backlog.
- **No delete-user endpoint —** users cannot be removed yet; the audit log would hold dangling references. When the endpoint ships it will use a set-null-on-delete policy so the historical trail is preserved.
- **Pagination capped at fifty rows —** audit log and evaluation run listings return at most fifty rows server-side. Fine at current scale; cursor-based pagination is documented for the first time the row count passes one thousand.
- **Audit chain is self-anchored —** the hash chain verifies against itself but is not yet anchored to an external write-once log. A production deployment would publish the daily root to Sigstore or a similar service.

- **Drift threshold is heuristic —** the seventy-five percent quality floor is a reasonable starting value, not a statistically derived one. Eight weeks of baseline data and a two-sigma analysis would replace the heuristic with a calibrated threshold.

> **Why say this out loud**
> An honest limitations section earns more trust than a flawless demo. Every simplification here is a deliberate choice, every one has a clear next step, and none of them breaks the core thesis of the platform.

13. Future Improvements

The limitations above map directly onto a roadmap. Everything here is additive — the core design does not need to change.

Next three to six months

- Migrate the database from SQLite to PostgreSQL with versioned schema migrations.
- Move background jobs to a distributed worker queue (Celery with Redis).
- Add single sign-on through SAML or OIDC.
- Introduce per-team row-level access so organisations can isolate their services.
- Add an email and Slack notification channel for critical alerts.

Next year

- Cross-vendor judge (OpenAI or Gemini) cross-validating a sample of runs alongside the existing Haiku judge, to fully break the single-vendor scoring correlation that a same-family Sonnet-plus-Haiku pairing only partially addresses.
- HIPAA, SOX, and GDPR compliance templates for out-of-the-box regulated workloads.
- Mobile triage app for on-call engineers to review AI drafts from a phone.
- External anchoring of the audit chain — publish the daily Merkle root to Sigstore or an equivalent transparency log so the chain can be verified against an outside witness.
- Replace the heuristic seventy-five percent drift floor with an eight-week baseline calibration and a two-sigma alert threshold derived from real data.
- Move from single-approver (Level 1 HITL) to a four-eyes contract where the drafter and the approver must be different users.
- Ability to register services for non-Anthropic models (OpenAI, Gemini, self-hosted) through a provider-abstracted client.

How the system can scale

The architecture is ready for scale. The database and scheduler are the two pieces that need to swap out; every other component already works as it will in production. The one AI entry point means a single code change flips the system from Anthropic-only to multi-provider. Audit chain integrity costs the same at one million rows as at one thousand — verification is still linear.

14. Quick Summary (Revision Section)

Use these ten points for quick revision before the presentation or for a fast overview of the system.

1. The platform is a single web app that monitors AI services, evaluates their quality, investigates incidents, and produces compliance reports.
2. It solves four gaps: no runtime view, drift caught too late, investigations in chat, and compliance stitched from many tools.
3. Four modules: Service Registry, Monitoring and Evaluation, Incidents and Maintenance, Governance and Compliance.
4. Quality is measured by running a small exam against each service; a second, smaller AI model (Claude Haiku) grades the answers — different from the Sonnet actor being scored — and drift is detected two ways.
5. Every AI-generated artifact is a draft. A named human must approve it with a note of at least twenty characters before it becomes official; the rule is enforced at three layers (browser, request boundary, handler).
6. Every AI call passes through six safety stages: sensitivity check, two-layer input scan (regex plus Haiku LLM classifier, fail-open on classifier outage), budget reserve (sixty per minute, forty per user, five dollars a day, twenty-five a month), API call, output scan, and audit persist.
7. Every action writes to a cryptographically chained audit log; one button verifies the entire chain and reports any tampering.
8. Three roles govern access: Admin, Maintainer, Viewer. Every denied attempt is itself logged, so misuse is visible.
9. The platform is a reference implementation — SQLite and an in-process scheduler are deliberate simplifications that swap cleanly for production components.
10. Approximately one hundred and fifty-eight automated tests across nineteen files run in under a minute; coverage is around seventy-eight percent and the build fails if it drops below a sixty-five percent floor.
11. The thesis in one line: one schema, one audit chain, one human-in-the-loop contract.

Bonus — How to Explain This System in 1–2 Minutes

For a presentation opening, an interview, or a quick answer in Q&A, this is the story in under two minutes.

> **The elevator version**
> We built a centralised AI operations platform. Teams running AI services in production have no single place to see how those services are behaving — uptime is tracked, behaviour is not. Our platform gives them one dashboard for quality, a structured incident workflow when something breaks, and a tamper-evident audit trail that an auditor can trust. A two-tier model economy uses Sonnet as the actor and Haiku as the judge plus injection classifier, so every answer is graded by a different model from the one that produced it. Every AI-generated summary, draft, or insight must be reviewed and approved by a named human with a note. Every action is cryptographically chained. One schema, one audit chain, one human-in-the-loop contract. It is a reference implementation with approximately 158 automated tests at 78% coverage, so every claim the platform makes is backed by a test that proves it works.

Shorter versions

The thirty-second version

It is a single web app that lets a team monitor their AI services, catch drift when quality drops, investigate incidents with AI-drafted summaries that a human must approve, and export a tamper-evident compliance report an auditor can trust — all in one place with one audit chain.

The one-sentence version

A single-pane AI operations control room where every action is gated, every denial is audited, every AI draft needs a human to sign off, and every change is cryptographically chained.

For a Q&A answer

When asked “what does your platform actually do?” start with the problem: “Teams running AI features have no single place to see whether their AI is still behaving correctly. Uptime does not mean quality.” Then give the solution: “Our platform is one web app that covers monitoring, evaluation, incidents, and compliance — with a two-tier model economy where Haiku judges what Sonnet produces, a human-in-the-loop contract on every AI output, and a tamper-evident audit chain.” Finish with the proof: “Approximately one hundred and fifty-eight automated tests back the platform’s claims. A single button verifies the audit chain is intact. A single date range produces a compliance report an auditor can read.”

Points you can always fall back on

- One schema, one audit chain, one approval contract — across every module.
- The AI drafts. The human approves. Every time.
- Every claim is backed by a specific test.
- An auditor's view is one PDF and one verify button.
- Defence in depth — no single safety check carries the whole weight.